

The **Android AI Runtime** — Technical Blueprint

A systems design for the local AI runtime that should ship with every Android device: a shared, secure, OpenAI-compatible inference service that any app can call — `EdgeLM.chat(...)` — without embedding an engine or touching the cloud.

Author: Founding Engineer / CTO

Audience: Architecture Review

Status: Draft for review

Scope: 16 parts

PART 01 — PRODUCT VISION

What Android is missing

Every Android phone is a supercomputer with a GPU, an NPU, and gigabytes of RAM — yet there is no operating-system primitive for running AI. Each app that wants intelligence ships its own inference engine, downloads its own multi-gigabyte weights, drains the battery independently, and either runs slowly or ships user data to a cloud. EdgeLM proposes the missing primitive: **a first-class AI runtime, owned by the platform, shared by every app.**

Mission

Make on-device AI a native capability of Android — a system service any app can call, so intelligence is as ordinary to request as location or storage.

Vision

A world where AI runs where data lives. EdgeLM becomes the default local runtime across phones, tablets, wearables, cars, and glasses — the layer between silicon and software, invisible and everywhere.

Product philosophy

Five convictions drive every decision in this document:

- 1 **Inference is infrastructure, not a feature.** It belongs in the OS layer, like the graphics stack or the media codec service — not re-implemented in every app.
- 2 **The model is a shared, mmap'd resource.** If ten apps use Llama-3.2-3B, there is one copy of the weights in memory, one download on disk, one warm runtime. Anything else wastes the device.
- 3 **The device is the datacenter; the cloud is a fallback.** Local-first is not a privacy slogan — it is the default execution target. Cloud is an explicit, opt-in overflow.
- 4 **Compatibility is a growth strategy.** Speak the API developers already know (OpenAI) so migration is one line, then earn lock-in through quality, not incompatibility.
- 5 **The runtime must be invisible when it works and legible when it doesn't.** Developers get world-class tracing, profiling, and benchmarks; users get a calm system UI that shows exactly which app used which model, and why the battery moved.

Long-term direction

EdgeLM starts as an installable runtime + SDK, matures into a developer platform (Hub, Studio, CLI, marketplace), and ends as an **ambient intelligence layer** spanning every edge device class. The endgame is that "add AI to your app" means "call EdgeLM," the way "store a file" means "call the filesystem." The strategic bet: whoever owns the runtime API owns the developer relationship for on-device AI for the next decade.

DX philosophy

Five minutes to first token. One import, typed APIs, sane defaults, zero model-format archaeology. The runtime hides quantization, backend selection, memory, and scheduling. Raycast-grade polish in every tool.

Privacy philosophy

Privacy is topology, not policy. Prompts never leave the device by default; there is no telemetry on content, ever. Cross-app isolation is enforced by the kernel, not by good behavior. The user can see and revoke everything.

Performance philosophy

Measure everything; optimize the p95, not the demo. Every millisecond of latency and every milliamp of battery is a budget line. Shared caches, warm runtimes, and hardware acceleration are the default, not a "pro" mode.

◆ PROACTIVE OPPORTUNITY – THE "AI CAPABILITY" AS AN OS CONTRACT

Beyond a runtime, EdgeLM should define a **capability manifest**: an app declares in its manifest which modalities and models it needs (`llm.chat`, `vision.caption`, `speech.transcribe`). The OS/EdgeLM can then pre-provision models, show honest permission prompts, and let the Play Store surface "AI requirements" the way it surfaces permissions today. This turns EdgeLM from a library into a platform contract — the real moat.

PART 02 – SYSTEM ARCHITECTURE

One runtime, many apps, shared metal

EdgeLM runs as a persistent, sandboxed **system service** (a bound Android `Service`, aspirationally a platform service) that owns the models, the memory, the GPU/NPU scheduling, and the security boundary. Apps never touch the engine; they talk to a thin SDK that marshals calls over Binder to the service, which dispatches to native inference backends. Think *Play Services for AI*, with *Kubernetes-style scheduling* underneath.

🏠 Application Layer — untrusted, sandboxed per app

notes.app · camera.app · keyboard.app · your.app

EdgeLM.chat()

POST /v1/chat/completions

capability manifest

▼ in-process SDK call

△ SDK Layer — Kotlin/Java + REST client, per-app

typed API · streaming · token accounting · ret/backoff · local loopback server

AIDL stubs

Flow<Token>

OpenAI shim

▼ Binder IPC (AIDL) · shared-memory for tensors

💎 EdgeLM System Service — trusted, single instance, isolated process

the brain: request broker + policy + lifecycle

API Gateway

Security / Permission Broker

AI Scheduler

Model Lifecycle Mgr

Memory Manager

Plugin Broker (MCP)

Metrics / Tracing

▼ JNI · zero-copy tensor handoff

⚙️ Native Runtime Layer — C/C++/Rust, hot path

backend abstraction · graph exec · KV/prompt cache · quant kernels · sampler

llama.cpp

MLC / TVM

ONNX Runtime

Vulkan

NNAPI

QNN

tokenizers

▼ vendor drivers

📁 Hardware Layer

CPU (big.LITTLE) · GPU (Adreno/Mali/Xclipse) · NPU (Hexagon/APU/Exynos) · DSP

📁 Model Store (mmap files + signed manifest)

📁 Shared Memory Arena (weights + KV pools)

📁 Encrypted Cache (prompt / KV persistence)

Fig 2.1 — EdgeLM layered architecture. The system service is the single trust and scheduling boundary; models and memory are shared across all apps below it.

Why a system service, not a library

DECISION — CENTRAL SERVICE MODEL OVER PER-APP EMBEDDING

EdgeLM runs inference in **one shared, out-of-process service**, not as a library linked into each app. This is the foundational architectural choice.

TRADE-OFF ANALYSIS

Wins: one copy of weights in RAM and on disk (a 3B model at Q4 is ~2GB — embedding it in five apps is 10GB, sharing it is 2GB); global scheduling so apps can't collectively melt the SoC; one security boundary; centralized updates; battery/thermal governance across the whole device.

Costs: IPC latency on the request path (mitigated by shared memory for tensors — we never copy weights or KV over Binder); a single point of failure (mitigated by a supervised, auto-restarting service with request replay); versioning complexity (the service must support multiple model/ABI versions concurrently). We accept these because the RAM and scheduling wins are existential on mobile.

Component responsibilities

COMPONENT	RESPONSIBILITY	KEY DESIGN NOTE
API Gateway	Terminates Binder + local HTTP; validates, normalizes to internal IR, enforces quotas	OpenAI-shape in, canonical request out
Permission Broker	Maps calling UID → granted capabilities; gates every request	Ties into Android's permission + package manager
AI Scheduler	Orders, batches, preempts, and cancels inference jobs across apps	OS-style scheduler (Part 8)
Model Lifecycle Mgr	Download, verify, load, warm, evict, roll back models	State machine in Fig 2.2
Memory Manager	mmap arenas, KV/prompt pools, pressure-driven eviction	Part 4
Native Runtime	Backend selection + graph execution on the chosen accelerator	Part 3
Plugin Broker	Brokers tool-calls / MCP between models and sandboxed plugins	Part 9
Metrics/Tracing	Per-request spans, tokens/sec, energy, thermal — no content	Feeds DevTools (Part 13)

Model lifecycle state machine



Fig 2.2 – Model lifecycle. Weights become **Resident** via mmap without consuming committed RAM; the pages fault in lazily on first **Warm**. Eviction returns to Resident (mmap intact) or fully Evicted under pressure.

The crucial subtlety: **Resident** $\neq$ **RAM**. An mmap'd model on disk is "resident" (addressable) but costs almost no committed memory until pages fault in. Warming touches the pages the first token needs; the OS page cache does the rest. This lets EdgeLM keep dozens of models "resident" and pay real memory only for what's warm.

PART 03 – RUNTIME ENGINE

A backend-agnostic inference core

The Android hardware landscape is brutally fragmented: Adreno/Mali/Xclipse GPUs, Hexagon/Exynos/MediaTek NPUs, and a long tail of CPU-only devices. No single inference engine wins everywhere. EdgeLM's runtime is therefore an **abstraction layer over multiple backends** with a runtime-measured selection policy — not a bet on one engine.

Backend abstraction

Every backend implements one internal interface — `load(model)`, `prefill(tokens)`, `decode(kv)`, `caps()`, `profile()`. The core compiles a request into a canonical execution plan; the selected backend executes it. Adding hardware later means adding a backend, not touching the API.

llama.cpp – GGUF, broad coverage, CPU+Vulkan

MLC / TVM – compiled kernels, top GPU throughput

ONNX Runtime – vision/speech/embeddings, NNAPI EP

Vulkan – portable GPU compute

NNAPI – vendor NPU abstraction (legacy)

Qualcomm QNN – Hexagon direct

MediaTek NeuroPilot – APU

Samsung ENN – Exynos NPU

CPU (GGML/XNNPACK) – universal fallback

DECISION – MULTI-BACKEND, NOT SINGLE-ENGINE; LLAMA.CPP + MLC AS LAUNCH PAIR

Ship with **llama.cpp** (unmatched model/format coverage, reliable CPU + Vulkan) as the compatibility floor and **MLC/TVM** (compiled, quantization-aware kernels) as the performance ceiling on supported GPUs. Layer vendor NPU backends (QNN first — largest install base via Snapdragon) as opportunistic accelerators.

TRADE-OFF – WHY NOT NNAPI-ONLY, OR ONE ENGINE?

NNAPI promised a unified NPU abstraction but delivers inconsistent operator support, poor LLM-shaped workload performance, and is being de-emphasized in favor of vendor SDKs — so it's a fallback, not a foundation. A single engine (e.g., MLC everywhere) maximizes performance on flagships but strands the mid/low tier and every unusual model format. Multi-backend costs us a maintenance and testing matrix (mitigated by the shared interface + a conformance test suite every backend must pass), but it's the only way to cover the real device distribution.

Backend selection — how EdgeLM chooses

Selection is **measured, cached, and adaptive**, not hardcoded per SoC. On first run for a given (model, device) pair, EdgeLM executes a short micro-benchmark; results are cached in the model manifest and refreshed opportunistically.

```
// backend selection – decision order
fun selectBackend(model, device, request):
    candidates = backends.filter { it.supports(model.arch, model.quant) }
    candidates = candidates.filter { it.fitsMemory(model, device.freeRam()) }

    // 1. hard constraints: correctness & memory first
    if candidates.isEmpty() → return CPU_FALLBACK // always works

    // 2. policy: honor request hints (latency vs energy)
    val weight = when (request.priority, device.onBattery(), device.thermal()):
        FOREGROUND, *          → { tps: 0.7, energy: 0.1, ttft: 0.2 }
        BACKGROUND, battery    → { tps: 0.2, energy: 0.6, ttft: 0.2 }
        *, thermal > WARN       → { tps: 0.3, energy: 0.5, ttft: 0.2 } // prefer NPU/CPU, throttle GPU

    // 3. score against cached micro-benchmark profiles
    return candidates.maxBy { b -> score(b.profile(model, device), weight) }
```

The energy term is why an NPU often wins on battery even when a GPU is faster: NPUs deliver dramatically better tokens-per-joule. The scheduler (Part 8) can also *switch backends mid-session* if thermal state changes — decoding can migrate from GPU to NPU/CPU when the SoC heats up, because the KV cache is backend-portable (a deliberate constraint on our cache format).

GPU / accelerator pipeline

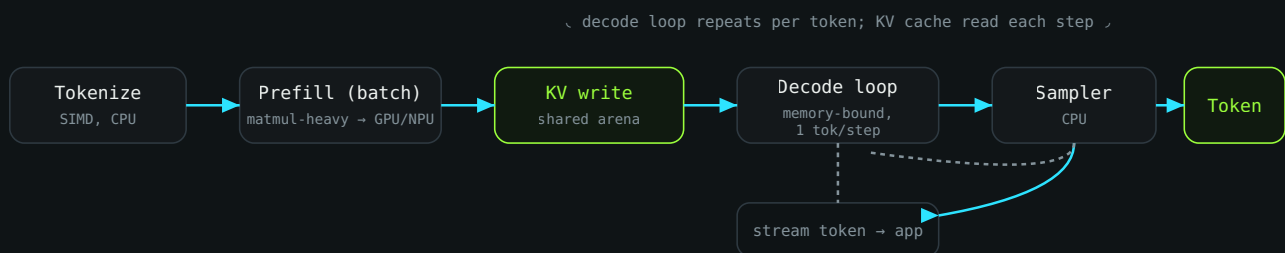


Fig 3.1 – Execution pipeline. **Prefill** is compute-bound (parallel matmuls → best on GPU/NPU); **decode** is memory-bandwidth-bound (one token at a time → often CPU/NPU-competitive). EdgeLM can place the two phases on *different* accelerators.

◆ PROACTIVE OPTIMIZATION – PHASE-SPLIT PLACEMENT

Because prefill and decode have opposite bottlenecks, EdgeLM treats them as separately schedulable stages: run prefill on the GPU (soak up the parallelism) and decode on the NPU or big-core CPU (bandwidth-efficient, cooler, better tokens/joule). Few runtimes do this today; it's a meaningful latency *and* battery win.

Runtime optimization & future hardware

- **Ahead-of-time compilation cache:** MLC kernels compiled per SoC are cached on-device and shared via Hub, so users don't pay compile time.
- **Operator fusion & graph rewriting** at load time; speculative and batched decode (Part 11).
- **Capability probing:** new SoCs are onboarded by a backend + a conformance run; unknown NPUs degrade gracefully to Vulkan/CPU rather than failing.
- **Forward path:** a stable backend ABI means EdgeLM can adopt future accelerators (dedicated LLM blocks, on-die SRAM caches) without SDK changes — the whole point of the abstraction.

PART 04 – MEMORY ARCHITECTURE

The real constraint is RAM, not FLOPs

On a phone, memory — not compute — is the wall. A mid-range device has 6–8GB shared between the OS, foreground app, and everything else; the Android low-memory killer is always watching. EdgeLM's memory manager exists to run the largest possible model at the highest possible throughput while staying invisible to the LMK. Every technique below serves one goal: **minimize committed RAM, maximize token throughput.**

Foundations: mmap + lazy loading

Weights are never `read()` into a heap buffer. They are `mmap()`'d directly from the model file, so the kernel page cache *is* the model's memory. Consequences we exploit:

- **Zero-copy, instant "load":** mapping a 2GB model is $O(1)$; pages fault in on demand during warm-up.
- **Free sharing:** two apps using the same model map the same physical pages — the OS deduplicates for us. This is the mechanism behind "one copy in RAM."

- **Reclaimable under pressure:** clean mmap'd pages can be dropped by the kernel and re-faulted later — the model degrades to slower, not dead, under memory pressure.

The caches that make decode fast

CACHE	WHAT IT HOLDS	DESIGN DECISION
KV cache	Per-session attention keys/values; the dominant runtime memory cost	Pooled + paged (see below); backend-portable format
Prompt cache	Prefill state for reused prefixes (system prompts, RAG context)	Content-addressed by prefix hash; shared across sessions of the same app
Compiled-kernel cache	AOT GPU kernels per SoC	On disk, shared via Hub
Tokenizer/vocab	Merges + vocab	mmap'd, shared per model

Paged KV cache — "virtual memory for attention"

Naive KV allocation reserves worst-case contiguous memory per session (huge and wasteful). EdgeLM allocates KV in **fixed-size blocks from a shared pool** (a PagedAttention-style design), mapping logical positions to physical blocks. This yields near-zero fragmentation, lets many sessions share one pool, and enables **copy-on-write prefix sharing** — ten chats with the same system prompt share those KV blocks until they diverge.

```
// KV block pool – shared across sessions & apps (with isolation tags)
Pool: N blocks × 16 tokens × d_model × 2(K,V) × dtype
session.kv = [blk#12, blk#7, blk#40, ...] // logical→physical map
sharedPrefix("You are a helpful assistant") → [blk#3, blk#4] // COW, refcount=10
onDiverge(session) → copyOnWrite(blk) // split only the changed block
```

The full memory toolkit

Context compression

Old KV entries are summarized/quantized (e.g. KV → 4-bit, or attention-sink + sliding window) to extend effective context without linear memory growth. Lossy but bounded; policy per model.

Cache reuse / dedup

Prompt-cache hits on shared prefixes skip prefill entirely — the single biggest latency win for chat and RAG. Content-addressed so identical contexts collapse.

Speculative decoding

A tiny draft model proposes k tokens; the target model verifies them in one batched pass. 1.5–3× decode speedup when acceptance is high — and the draft model is itself a shared resident model.

Adaptive quantization switching

Under memory/thermal pressure, hot-swap to a lower-precision variant of the same model (Q4→Q3) mid-session, reusing the KV cache. Graceful degradation instead of OOM.

Thermal adaptation

Memory bandwidth throttles when the SoC is hot; the manager reduces batch size, shrinks KV window, and can migrate decode to a cooler unit rather than stalling.

Background unloading

Idle models decay Warm→Resident→Evicted on a timer + pressure signal. Foreground app's model is pinned; background apps' models are the first to shrink.

DECISION — ONE SHARED ARENA WITH PER-APP ISOLATION TAGS

Weights and KV pools live in a **single service-owned arena** so physical pages are shared, but every block carries an owner/isolation tag. Cross-app KV sharing is allowed *only* for identical, non-sensitive prefixes (e.g. a public system prompt) and never for user content. This gets the RAM savings of sharing without leaking prompts between apps — the central privacy/perf tension, resolved at the block level.

TRADE-OFF — SHARING VS ISOLATION

Maximum sharing (dedupe everything) would leak one app's context into another's cache-timing and content. Maximum isolation (nothing shared) wastes the RAM that makes the whole product viable. We draw the line at **weights + public prefixes = shared; user KV = isolated & encrypted at rest**. Prompt-cache entries are namespaced by app UID unless explicitly marked shareable.

◆ PROACTIVE OPTIMIZATION – PREDICTIVE RESIDENCY

EdgeLM observes per-app usage patterns (with on-device, privacy-preserving stats) and **pre-warms** the model an app is statistically about to use — e.g. warm the keyboard's small LM when the keyboard gains focus, evict it when it loses focus. This turns cold-starts into warm-starts for the common case, using the scheduler's idle budget.

PART 05 – LOCAL API

Familiar on the surface, native underneath

The API has two faces. To developers it looks like OpenAI — a decision that makes migration a one-line base-URL change and gives us thousands of compatible tools for free. Underneath, the SDK talks to the service over **Binder + shared memory**, not HTTP, because on-device IPC has different physics than a network call.

Transport: REST vs gRPC vs Binder vs Unix sockets

TRANSPORT	LATENCY / COPIES	FIT FOR EDGELM
Local HTTP/REST (loopback)	TCP + JSON serialize both ways; kernel copies	Compat surface only — expose it, don't build on it
gRPC (HTTP/2)	Better streaming, still TCP + protobuf copies	Good for a desktop/localhost story; overkill on-device
Unix domain socket	No TCP stack; still byte-stream serialize	Reasonable fallback where Binder is unavailable
Binder + ashmem/AHardwareBuffer	Kernel-mediated IPC, zero-copy for tensors/images , UID identity built in	Primary transport — it's how Android services are meant to talk

DECISION – BINDER PRIMARY, LOCAL HTTP AS A COMPATIBILITY SHIM

The SDK marshals calls over **Binder (AIDL)**: it's the native Android IPC, carries the caller's UID for the permission broker automatically, and lets us pass large payloads (images, audio, embeddings) via shared-memory handles with **zero copies**. We *also* run a localhost HTTP server implementing the OpenAI shape, bound to `127.0.0.1` and gated by the same permission broker, so existing OpenAI SDKs, cURL, and cross-language apps work unmodified.

TRADE-OFF — TWO TRANSPORTS TO MAINTAIN

Supporting both doubles the surface, but the HTTP shim is a thin adapter over the same internal request IR — the real logic lives once, behind the gateway. The alternative (HTTP-only) would copy every image and audio buffer through the TCP stack and lose the free UID identity Binder gives us for security. Worth it.

Request sequence — a streaming chat call

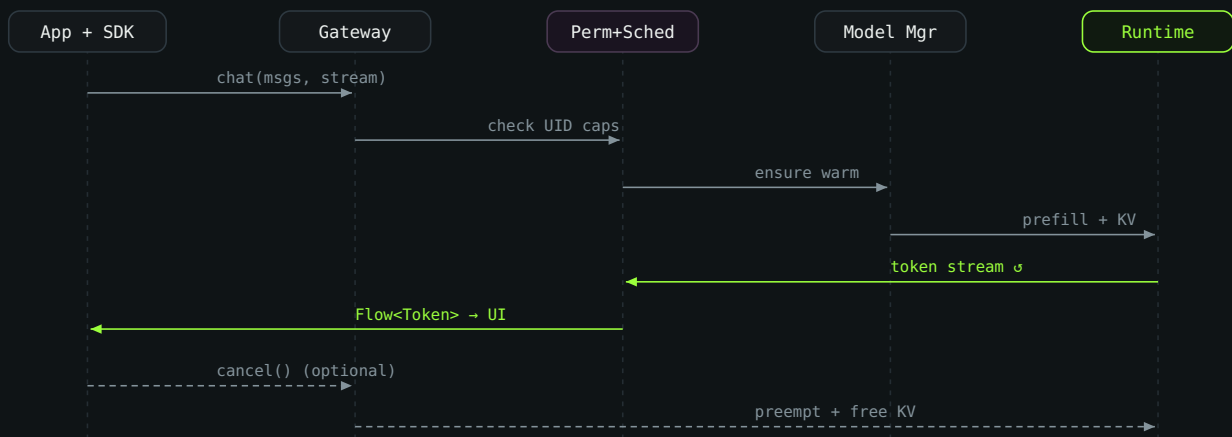


Fig 5.1 — Streaming chat sequence. Cancellation propagates all the way to the runtime, which frees the KV blocks immediately — critical for a device where memory is scarce and users kill requests often.

Surface area & formats

OpenAI-compatible where a counterpart exists; EdgeLM-native namespace (`/v1/edge/*`) for device concerns that OpenAI has no concept of (benchmarking, memory, model residency).

ENDPOINT	SHAPE	NOTES
<code>/v1/chat/completions</code>	OpenAI	Streaming (SSE), tools, JSON mode, vision (image parts)
<code>/v1/responses</code>	OpenAI Responses	Stateful, tool-loop, agent-friendly; server keeps KV for the session
<code>/v1/embeddings</code>	OpenAI	Batched; returns float or int8 vectors
<code>/v1/audio/transcriptions</code>	OpenAI	Speech-to-text; audio via shared-mem handle over Binder
<code>/v1/audio/speech</code>	OpenAI	TTS; streamed PCM/opus
<code>/v1/images/generations</code>	OpenAI	Diffusion backend (Part 12), gated by device tier
<code>/v1/edge/tokenize</code>	native	Token count/echo — lets apps budget context client-side, free
<code>/v1/edge/models</code>	native	List/pull/warm/evict; residency + quant variants
<code>/v1/edge/bench</code>	native	Run the micro-benchmark; returns tps/ttft/energy per backend
<code>/v1/edge/health</code> · <code>/metrics</code>	native	Liveness, warm set, mem/thermal state; Prometheus-style metrics (no content)

```
POST /V1/CHAT/COMPLETIONS · STREAMING, TOOL-ENABLED{
  "model": "llama-3.2-3b-instruct",
  "stream": true,
  "messages": [{"role":"user","content":"What's on my calendar tomorrow?"}],
  "tools": [{"type":"function","function":{"name":"calendar.query"}}],
  "edge": { // EdgeLM extension block – ignored by OpenAI clients
    "priority": "foreground", "max_energy_mj": 1500,
    "allow_backends": ["npu","gpu","cpu"], "cache": "prefix"
  }
}
// → SSE: data: {"choices":[{"delta":{"content":"You "}}]} ... [DONE]
```

◆ PROACTIVE ADDITION – THE `EDGE{}` EXTENSION ENVELOPE

Rather than fork the schema, EdgeLM tucks device-specific controls (priority, energy budget, backend hints, cache policy) into an optional `edge` field that vanilla OpenAI clients ignore. Full compatibility *and* full control, no schema war.

Make the hard parts disappear

The SDK's job is to make on-device AI feel like any other Jetpack API: coroutine-native, lifecycle-aware, typed, and boring in the best way. A developer should never see Binder, mmap, quantization, or backend names unless they ask.

```
KOTLIN · IDIOMATIC, COROUTINE-FIRST // one-time init — binds to the service, negotiates capabilities
EdgeLM.initialize(context)

// streaming chat — returns a cold Flow, cancels with the coroutine scope
val stream = EdgeLM.chat("llama-3.2-3b") {
    system("You are concise.")
    user(prompt)
    tools(CalendarTool, MapsTool) // registered plugins, brokered securely
    priority = Priority.FOREGROUND
}
stream.collect { token -> ui.append(token) } // backpressure honored

// other modalities — same ergonomics
val vec    = EdgeLM.embed(listOf(docA, docB)) // List<FloatArray>
val text   = EdgeLM.transcribe(audioUri)      // speech → text
val reply  = EdgeLM.chat(...).awaitText()     // non-streaming convenience

// capabilities & models are observable state, not imperative calls
EdgeLM.models().observe { catalog -> ... } // StateFlow of residency
if (!EdgeLM.permissions().has(Capability.CHAT))
    EdgeLM.permissions().request(Capability.CHAT) // launches system consent UI
```

Lifecycle-aware

Streams are bound to coroutine scopes and Android lifecycles; navigating away cancels inference and frees KV automatically. No leaked jobs, no zombie GPU work.

Declarative models

`ensureModel()` expresses intent; the SDK negotiates the best quant/backend for the device. Developers pick a model *family*, not a file.

Graceful capability tiers

`EdgeLM.deviceTier()` lets an app adapt: offer a 7B experience on a flagship, a 1B on a budget phone, cloud fallback where allowed.

DECISION — THIN SDK, THICK SERVICE

The SDK holds **no inference logic and no weights** — it's a typed Binder client plus ergonomics (~a few hundred KB). All heavy machinery lives in the shared service. This keeps app size tiny, lets the runtime improve without app updates, and guarantees every app benefits from one place getting faster.

TRADE-OFF — DEPENDENCY ON A SYSTEM SERVICE BEING PRESENT

A thin SDK means the app is useless if the service is absent (old device, not installed). We mitigate with (a) a Play-delivered service with an install-on-demand flow the SDK triggers, and (b) an optional embedded micro-runtime for a single small model as a last-resort fallback. Default path assumes the shared service; the fallback trades RAM for availability.

PART 07 — SECURITY & PERMISSION MODEL

Untrusted apps, private prompts, shared silicon

EdgeLM sits between mutually-untrusting apps and a shared resource that touches the user's most sensitive data (their prompts) and their most contended resources (RAM, GPU, battery). The security model must prevent one app from reading another's prompts, monopolizing the SoC, or draining the battery — while integrating cleanly with Android's existing permission machinery rather than inventing a parallel universe.

Capability-based permissions, brokered by the OS

EdgeLM defines **AI capabilities** as first-class permissions, declared in the manifest and granted through the standard Android consent flow — the same mental model as camera or location.


```

ANDROIDMANIFEST.XML<uses-permission android:name="ai.edgelm.CHAT" />
<uses-permission android:name="ai.edgelm.EMBED" />
<uses-permission android:name="ai.edgelm.VISION" />
<uses-permission android:name="ai.edgelm.BACKGROUND_INFERENCE" /> // higher-scrutiny
<edgelm:capability model-class="llm.small" max-context="8k" />

```

Who approves what

Low-risk read-only capabilities (CHAT, EMBED with the app's own data) are grant-on-first-use with a clear prompt. High-risk ones (BACKGROUND_INFERENCE, plugin access to Calendar/Files, cloud fallback) require explicit, revocable, per-app consent — surfaced in a system "AI permissions" page.

Identity via Binder UID

Every request carries the kernel-verified caller UID; the permission broker maps UID → package → granted capabilities. An app cannot spoof another's identity because Binder, not the app, stamps the UID.

Isolation & sandboxing

- **Process isolation:** the service runs in its own process/UID (`isolatedProcess` where possible). Apps never share an address space with the engine or with each other.
- **Prompt isolation:** per-app KV and prompt caches are namespaced by UID and **encrypted at rest** with keys in the Android Keystore / StrongBox. One app's context is never addressable by another.
- **Side-channel hardening:** shared weights are read-only and public, so sharing them leaks nothing; user-derived state is never shared. Cache-timing on shared prefixes is limited to public content only.
- **Resource sandboxing:** per-app quotas on concurrent requests, tokens/sec, GPU time-slices, and an **energy budget** (millijoules/hour) enforced by the scheduler. A malicious app hits its quota and is throttled, not the device.

DECISION — INTEGRATE WITH ANDROID PERMISSIONS, ADD AN ENERGY BUDGET PRIMITIVE

We reuse Android's permission + Keystore + package infrastructure (trust, not reinvention) and add one genuinely new primitive Android lacks: a **per-app compute/energy budget** for AI, because unbounded local inference is a novel battery-abuse vector the platform doesn't yet police.

TRADE-OFF — CONSENT FATIGUE VS SAFETY

Too many prompts and users blind-tap "allow"; too few and abuse slips through. We tier by risk (silent for own-data reads, prompted for cross-app/background/cloud) and give a single legible control surface, accepting that the highest-value capabilities cost one deliberate consent.

◆ PROACTIVE – CONTENT FIREWALL FOR TOOL CALLS

When a model calls a plugin (e.g. `files.read`), EdgeLM can enforce a **data-flow policy**: content read under a "local-only" capability is tainted and cannot subsequently be sent to a cloud-fallback endpoint without a fresh, explicit user consent. This blocks the subtle exfiltration path where a local model launders private data to the cloud via tool output.

PART 08 – AI SCHEDULER

An operating-system scheduler for inference

When the keyboard, the browser, a background summarizer, and a foreground chat app all want the GPU at once, someone must decide who runs, in what order, at what cost. This is a scheduling problem, and the right references are CPU schedulers and Kubernetes — not request queues. EdgeLM's scheduler provides priorities, fairness, preemption, and battery/thermal awareness.

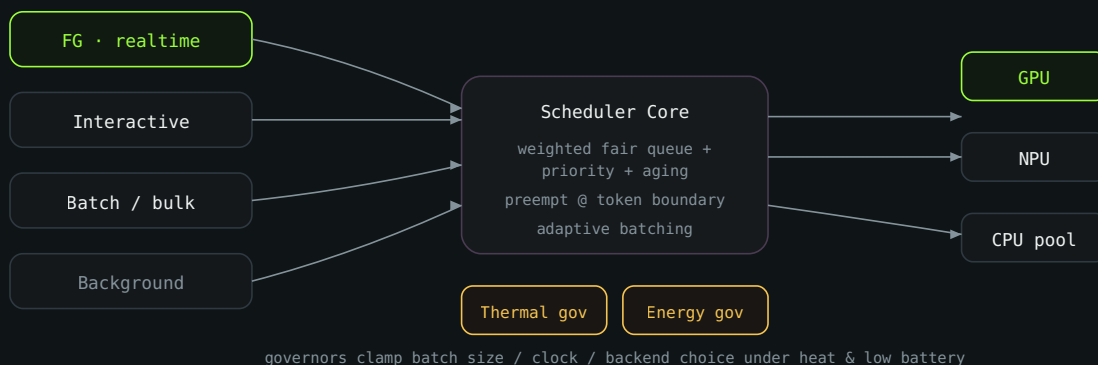


Fig 8.1 – Multi-class scheduler. Jobs enter priority classes; a weighted-fair core picks the next runnable job, preempts at token boundaries, and batches compatible jobs. Thermal/energy governors sit above the executors as hard clamps.

Algorithms & policies

- **Priority classes with weighted fair queuing:** foreground-interactive > interactive > batch > background. Within a class, weighted-fair share prevents any one app from starving peers.
- **Preemption at token boundaries:** decode is naturally step-wise — the scheduler can pause a low-priority stream *between tokens* (KV preserved) to admit a foreground request, then resume. No wasted compute, sub-100ms responsiveness.

- **Aging / starvation prevention:** a job's effective priority rises with wait time, so background work eventually runs even under sustained foreground load.
- **Adaptive continuous batching:** compatible decode steps (same model, same backend) are fused into one batched kernel launch, raising throughput; batch size shrinks under thermal pressure.
- **Foreground acceleration:** the app the user is looking at gets the GPU and a latency SLO; background jobs are steered to NPU/CPU and best-effort.
- **Battery & thermal awareness:** governors translate device state into scheduling constraints — on low battery, background inference is deferred or moved to the most energy-efficient backend; at thermal WARN, GPU slices shrink and work migrates.
- **Cancellation is first-class:** cancel frees KV and slice immediately (Fig 5.1) — essential because users abandon requests constantly on mobile.

DECISION — TOKEN-BOUNDARY PREEMPTION OVER TIME-SLICE PREEMPTION

Preempting mid-matmul is expensive and messy; preempting between generated tokens is nearly free because the model's state is exactly the KV cache. We make the token the scheduling quantum — an insight that only works because inference is iterative. This gives interactive latency without throughput loss.

TRADE-OFF — THROUGHPUT VS LATENCY VS BATTERY IS A LIVE TENSION

Aggressive batching maximizes tokens/sec but hurts p95 latency for interactive apps; energy-optimal placement can be slower. There is no single right answer, so these are **policy weights** (Part 3's selection) tuned per device state, not constants — and exposed to advanced apps via the `edge{}` envelope.

PART 09 — PLUGIN PLATFORM

Giving models safe hands

A runtime that can only generate text is a curiosity; one that can safely read your calendar, search your files, control your lights, and call an API is a platform. EdgeLM brokers **tool calls** between models and capabilities — calendar, camera, maps, browser, files, notes, home automation, external APIs, and standard **MCP servers** — through a permissioned, sandboxed boundary.

Two plugin classes

CLASS	RUNS WHERE	EXAMPLE
Capability plugins	In the <i>owning app's</i> process, exposed via a declared tool schema	An app exposes <code>notes.search</code> to models it invokes
System plugins	Sandboxed, EdgeLM-brokered, user-granted	Calendar, Files, Home — shared across apps with per-app consent
MCP servers	Local (in-sandbox) or remote (network-gated)	Any Model Context Protocol tool; EdgeLM is an MCP host

Secure communication & sandboxing

- **Everything is a brokered RPC, never a direct call.** A model emits a tool-call intent; the Plugin Broker validates it against the calling app's granted capabilities, executes it in the plugin's sandbox, and returns typed results. The model never touches the plugin directly.
- **Sandboxed execution:** system plugins run in isolated processes with their own permission set; a plugin that reads Files cannot open a socket unless separately granted. Least privilege by construction.
- **Schema-typed & validated:** tool inputs/outputs are JSON-schema-typed; the broker rejects malformed or out-of-policy calls before execution — a model cannot smuggle a `rm -rf` through a `calendar.query`.
- **Human-in-the-loop for side effects:** read tools can be auto-approved within consent; *write/actuating* tools (send message, unlock door, purchase) require a confirmation surface, with a per-app "always allow for this action" option.
- **Network policy:** remote MCP/external-API plugins are the only ones allowed egress, and only under an explicit capability; the data-flow firewall (Part 7) prevents laundering local data outward.

DECISION — ADOPT MCP AS THE NATIVE TOOL PROTOCOL

Rather than invent a proprietary tool interface, EdgeLM speaks **MCP** as a first-class host. This instantly makes the growing MCP ecosystem available on-device and means server authors write once for cloud and edge. We extend MCP only with an on-device *capability/consent* layer it lacks.

TRADE-OFF — POWER VS BLAST RADIUS

Tool calling turns a text model into an agent that can act — enormously useful, and the largest security surface in the system. We contain it with least-privilege sandboxes, typed schemas, consent for side effects, and taint-tracked data flow, accepting some friction on write actions as the price of safety.

Models that "just work" on this device

The single hardest thing about on-device AI today is picking and provisioning a model that fits *this* phone. Hub abstracts that away: developers and users request a model *family*; Hub resolves the right quantization, format, and backend build for the device, verifies it, and streams it down efficiently.

Verified & signed

Every model is content-addressed and cryptographically signed; the runtime refuses to load an unsigned or tampered artifact. Provenance and license metadata travel with the weights.

Hardware compatibility resolver

Hub knows the device SoC, RAM, and accelerators; it serves the variant that fits (e.g. Q4 for 8GB, Q3 + sliding-window for 4GB) rather than a one-size file.

Benchmarks & ratings

Community + first-party tps/ttft/energy numbers per device class, plus quality ratings — so choice is data-driven, not vibes. Powered by EdgeLM Benchmark (Part 13).

Delta updates

Model updates ship as binary deltas against the resident version; a fine-tune bump downloads MBs, not GBs. Critical on metered mobile data.

Rollback & pinning

Signed version history with instant rollback if a new model regresses; apps can pin a known-good version for reproducibility.

Automatic optimization

On publish, Hub runs quantization + per-SoC kernel compilation server-side, caching artifacts so users never pay compile time (ties to Part 3).

DECISION — RESOLVE MODELS LIKE A PACKAGE MANAGER, NOT A FILE HOST

Hub behaves like `apt`/Play for models: request `llm.small@latest`, get the right build for your device with dependencies (tokenizer, kernels) resolved and signed. This is the difference between "download a GGUF and pray" and a real platform.

TRADE-OFF — CURATION VS OPENNESS

A fully open registry maximizes selection but invites malicious/broken models; a closed one is safe but slow and gatekept. We run a **verified tier** (signed, benchmarked, scanned) plus an **open tier** (community, clearly labeled, sandboxed, unsigned-load requires developer opt-in) — trust by default, freedom on request.

PART 11 — PERFORMANCE ENGINEERING

Optimizations beyond today's runtimes

Because EdgeLM is a shared, long-lived, cross-app service — not a per-app library — it can exploit optimizations no single-app runtime can even attempt. The shared vantage point is the moat.

Cross-app shared KV & prompt cache

Common system prompts and RAG contexts are prefilled once and reused across apps (public content only). Unique to a central service.

Speculative + Medusa decoding

Shared draft models accelerate every app's decode; multi-head speculative decoding for 2–3× on accepted tokens.

Adaptive quantization

Precision follows pressure and priority — Q4 foreground, Q3 background, switchable mid-session over a shared KV.

Context deduplication

Content-addressed prefixes collapse identical contexts across sessions to one physical KV set (COW).

Predictive warm-up

Usage-pattern prediction pre-warms the model an app is about to need, converting cold starts to warm (Part 4).

GPU memory pooling

A single service-owned GPU allocator avoids per-app fragmentation and repeated buffer churn; buffers are recycled across requests.

Background AOT compilation

Kernels compile during charging/idle and cache to disk + Hub; users never wait on first run.

Runtime graph optimization

Fusion, constant folding, and layout selection at load, specialized to the chosen backend.

◆ NOVEL OPTIMIZATIONS WE SHOULD OWN

- 1 · **Phase-split placement** — prefill on GPU, decode on NPU/CPU (Part 3), a latency *and* battery win almost no runtime does.
- 2 · **Charge-time precompute** — when plugged in and idle, EdgeLM pre-embeds new documents, pre-warms predicted models, and pre-compiles kernels, so daytime interactions are instant and cool. Treat the charger as free compute.
- 3 · **Semantic KV compression** — evict/compress KV by learned importance (attention-sink + salience) rather than pure recency, extending effective context per MB.
- 4 · **Federated benchmark-driven autotuning** — anonymous, aggregated per-SoC performance telemetry (no content) continuously refines backend-selection profiles for the whole fleet.
- 5 · **Cross-request continuous batching by embedding-similarity** — batch not just same-model requests but semantically similar prefills to maximize kernel occupancy.

PART 12 – FUTURE MODALITIES & DEVICE CLASSES

One runtime, every modality, every edge

The architecture was designed so new modalities are new backends + endpoints, not new products. The `edge{}` envelope, scheduler, and memory manager are modality-agnostic.

MODALITY	HOW IT SLOTS IN	HORIZON
Vision (caption, VQA, OCR)	Vision encoder backend → image parts in chat; shared with LLM via projector	Near
Speech (ASR) & TTS	ONNX/whisper + TTS backends; streamed audio over shared-mem	Near
Embeddings + RAG	Embedding backend + on-device vector index as a system plugin	Near
Agents & tool calling	Responses API + MCP broker (Parts 5, 9)	Near
Image generation	Diffusion backend, device-tier gated; charge-time precompute friendly	Mid
Video understanding/gen	Streaming vision backend; heavy — flagship + charge-time only	Far
Wearables / AR glasses	Thin client to a phone-hosted runtime, or micro-runtime on-device; latency-critical scheduling class	Mid
Cars / IoT / robotics	Same backend ABI on non-Android edge; real-time class + safety sandboxing	Far

DECISION — COMPANION-OFFLOAD FOR CONSTRAINED DEVICES

Glasses and watches can't host large models. EdgeLM defines a **companion protocol**: the wearable's SDK transparently offloads to the paired phone's runtime over BLE/UWB, falling back to a tiny on-device model when disconnected. The developer writes one API; EdgeLM routes to the best available runtime in the user's personal device mesh.

◆ PROACTIVE — A PERSONAL, CROSS-DEVICE RUNTIME MESH

The long-term unlock isn't one device — it's the user's **fleet**. Phone, tablet, laptop, car, and glasses each advertise their runtime + resident models; EdgeLM schedules a request to whichever is best positioned (nearest data, most idle silicon, on charger), privately over the local network. This is the on-device answer to the cloud, and no one owns it yet.

PART 13 — DEVELOPER PLATFORM & TOOLING

Raycast-grade tools for on-device AI

Adoption is won or lost on the debugging loop. EdgeLM ships a full toolchain so developers can see exactly what the runtime is doing — tokens, memory, energy, backend choice — and fix it fast.

EdgeLM CLI

`edgelm pull / run / bench / profile / trace`. Scriptable, CI-friendly, the backbone of every other tool.

Desktop companion

Connect a device over ADB; mirror the runtime dashboard, stream traces, and hot-swap models during development.

Inspector

Live view of every in-flight request: prompt tokens, KV footprint, chosen backend, tokens/sec, and energy — per app.

Profiler

Flamegraphs of prefill vs decode, kernel timings, memory-bandwidth saturation, thermal correlation.

Benchmark suite

Reproducible model × device × backend runs feeding Hub's numbers; a public leaderboard developers trust.

Tracing & logs

OpenTelemetry-style spans across SDK → service → runtime; structured logs with zero prompt content by default.

VS Code extension

Inline token counting, model picker, prompt playground, and one-click "run on connected device."

Android Studio plugin

Manifest capability hints, model-size lint ("this model won't fit the min-spec you target"), and a live inference panel.

Replay & diff

Capture a session and replay it against a new model/quant to catch quality/perf regressions before shipping.

DECISION — TOOLING IS A LAUNCH FEATURE, NOT A V2

We invest in Inspector/Profiler/CLI *from day one*. On-device AI fails silently and weirdly (OOM, thermal throttle, wrong backend); without visibility, developers churn. World-class DX is the wedge against every lower-level competitor.

PART 14 — APPLICATION UX

The system app: Linear × Raycast × Material 3

The EdgeLM app is where users see and control the runtime. It must feel like a premium system utility — calm, dense, legible — not a consumer chatbot. The user's core questions: *what's using AI, what's it costing me, and can I trust it?*



- **Dashboard** — warm set, throughput, shared RAM, energy, top consumers at a glance.
- **Models / Downloads** — resident + available models, variants, delta-update progress, pin/rollback.
- **Benchmarks** — this device vs class; per-backend tps/tft/energy.
- **API Activity** — live request log per app (metadata only), with a kill switch.
- **Memory / Battery / GPU** — resource inspectors with history and per-app attribution.
- **Permissions** — the single control surface for AI capabilities and plugin grants; revoke anything.
- **Developer Mode** — traces, Inspector, backend override, verbose logs, unsigned-model opt-in.
- **Settings** — cloud-fallback policy, energy budgets, default quant tier, privacy controls.

DECISION — LEAD WITH RESOURCE TRANSPARENCY, NOT CHAT

The differentiating UX is **trust through visibility**: users can see exactly which app used which model, how much battery it cost, and cut it off. This is what makes a system-level AI service feel safe to grant broad permissions to — and it's the opposite of a black-box chatbot.

Open core, monetize the platform

Infrastructure earns trust by being open and earns revenue by being indispensable at scale. The runtime is free and open to win ubiquity; money comes from the layers around it.

LAYER	LICENSE / MODEL	RATIONALE
Runtime + core SDK	Apache-2.0, free forever	Trust anchor + adoption engine; must be forkable to be trusted as infra
EdgeLM Hub (verified tier)	Free for devs; rev-share on premium/licensed models	Marketplace take-rate scales with ecosystem
Studio / advanced DevTools	Free core; Pro subscription	Teams pay for collaboration, replay, fleet profiling
Cloud fallback	Metered usage	Local-first, cloud-billed overflow — aligned incentive (we profit when local can't)
Enterprise	Annual license + support SLA	Private registry, fleet policy, SSO, audit, compliance — the ACV driver
SDK for commercial embedding	Permissive; optional commercial support	No friction for indie devs; support contracts for those who want them

DECISION — NEVER MONETIZE THE INFERENCE PATH WITH ADS

The runtime that sees every prompt must never have an incentive to exploit them. Revenue comes from teams, scale, and the marketplace — **never** from content, ads, or selling telemetry. This is a strategic promise as much as an ethical one: it's what earns the permission to be the shared, trusted layer.

TRADE-OFF — OPEN RUNTIME INVITES FREE-RIDERS (INCL. OEMS)

An Apache runtime means Google or an OEM could bundle their own build. We treat that as distribution, not threat: our moat is Hub (models + benchmarks), DevTools, the cross-device mesh, and the developer relationship — none of which fork with the code. Openness maximizes the surface we monetize around.

From MVP to the Android standard

Sequenced to prove the hard technical bet first (shared runtime, real perf), then build the platform, then earn the standard. Each phase is gated on a measurable milestone, not a date.

PHASE 0 · MVP

Prove the shared runtime

Tech: single-service inference, llama.cpp + Vulkan, mmap sharing, Binder API, one flagship SoC. **Product:** demo app + Inspector. **Ecosystem:** 3–5 design-partner apps. **Gate:** two apps sharing one in-memory model at >30 tok/s.

PHASE 1 · BETA

Open the platform

Tech: scheduler, permission model, KV paging, MLC backend, OpenAI HTTP shim. **Product:** Hub v1, CLI, SDK 1.0, system app. **Ecosystem:** open-source runtime, public docs. **Gate:** 5-minute integration; first non-partner apps in production.

PHASE 2 · v1.0

Production-grade & multimodal

Tech: vision/speech/embeddings, MCP + tools, QNN NPU backend, thermal/energy governors, delta updates. **Product:** Studio, benchmarks leaderboard, marketplace. **Ecosystem:** hundreds of apps, model publishers. **Gate:** apps depend on EdgeLM in the Play Store.

PHASE 3 · ENTERPRISE & SCALE

Fleet, policy, cross-device

Tech: private registry, cloud fallback, device mesh (companion offload), federated autotuning. **Product:** Enterprise console, SSO/audit, support SLAs. **Ecosystem:** OEM & enterprise pilots. **Gate:** a regulated org standardizes on EdgeLM.

PHASE 4 · THE STANDARD

The default AI runtime for Android — and beyond

Tech: cross-platform backend ABI (wearables, auto, IoT), on-device fine-tuning, agent orchestration.

Product: ambient runtime mesh. **Ecosystem:** OEM pre-install, "Built on EdgeLM" as a badge. **Gate:** "add AI" means "call EdgeLM."

CLOSING THESIS

EdgeLM is a bet that **on-device inference is an operating-system responsibility**, not an application one — and that whoever builds the shared runtime, the model package manager, and the developer tooling first will own the platform layer for the next era of mobile computing. The technical wedge is memory sharing and scheduling that only a central service can do; the strategic wedge is OpenAI-compatibility and world-class DX; the moat is Hub, tooling, and the cross-device mesh. We ship the runtime open, earn ubiquity, and build the business around it.

EdgeLM · Engineering Design Document · Draft for architecture review

Forward-looking systems design. Performance figures, device names, and timelines are illustrative targets for planning, not measured results or commitments. "OpenAI-compatible" denotes API-shape compatibility only; EdgeLM is not affiliated with OpenAI, Google, Qualcomm, MediaTek, Samsung, Apple, or any named project. References to third-party engines (llama.cpp, MLC, ONNX Runtime, etc.) describe integration intent under their respective licenses.